

FPGA-Accelerated Histograms of Oriented Gradients for Human Detection

John Rottinghaus, Candice Liu, Andrew Gharthey

Abstract—Human detection is an increasingly important task in embedded computer vision, but achieving real-time performance is difficult when feature extraction is performed entirely in software. This paper presents an accelerated field-programmable gate array implementation of the Histograms of Oriented Gradients algorithm for human detection. The design optimizes multiple stages of the feature extraction pipeline, including gradient convolution, orientation binning, and cell histogram generation. Performance is improved through implementing rolling line buffers, histogram privatization, dataflow streaming, tile buffering, and bit-width reduction. The proposed accelerator is evaluated against a software baseline for performance analysis. Our results show a 75.6 times speedup compared to traditional software-only approaches while maintaining functional detection capability and low power consumption on a TUL PYNQ-Z2 board. These results demonstrate that targeted hardware acceleration can improve feature extraction efficiency for low-power embedded vision systems.

Index Terms— dataflow streaming, embedded computer vision, feature extraction, field-programmable gate array, FPGA, hardware acceleration, high-level synthesis, histogram of oriented gradients, HOG, human detection, line buffer

I. INTRODUCTION

Human detection is an important task in embedded computer vision systems, with applications in surveillance, robotics, autonomous vehicles, drones, biometrics, and human-machine interaction. In many of these applications, image processing must be performed in real time while operating under limited power, memory, and computing resources. Although modern deep learning methods are widely used for object detection, traditional feature extraction algorithms remain useful in constrained environments because they are interpretable, computationally structured, and suitable for hardware acceleration.

Histograms of Oriented Gradients (HOG) is a feature descriptor that represents local object shape by measuring the distribution of gradient directions within an image. The algorithm computes gradients for each pixel, divides the image into cells, accumulates orientation histograms for each cell, normalizes groups of cells into blocks, and forms a feature vector that can be used for classification. However, this pipeline can be computationally expensive when executed entirely in software. In the baseline software implementation used in this project, gradient convolution was identified to be the dominant bottleneck, accounting for around 97% of the total execution

time, with cell histogram generation introducing additional cost due to repeated memory access and histogram update dependencies.

Prior work has established HOG as an effective feature descriptor for human detection and has explored hardware-oriented optimizations for gradient computation, orientation binning, and histogram accumulation. However, many of these approaches focus on individual parts of the algorithm, such as the descriptor itself, convolution mapping, orientation approximation, or histogram memory contention. This project differs by combining multiple stage-level optimizations into a single field-programmable gate array-based implementation of the HOG pipeline.

This work presents a field-programmable gate array (FPGA) implementation for accelerating HOG-based human detection. The proposed design targets multiple bottlenecks in the feature extraction pipeline, including gradient convolution, orientation binning, and cell histogram generation. Performance is improved using rolling line buffers, division-free orientation binning, histogram privatization, dataflow streaming, tile buffering, bit-width reduction, and high-level synthesis (HLS) loop pipelining and unrolling. These optimizations reduce redundant memory accesses, improve pipeline throughput, and allow multiple feature extraction stages to execute parallelly in hardware. To evaluate detection accuracy, cosine similarity is calculated between HOG feature vectors of a reference image and test dataset, and human classification is determined based on each similarity score meeting a predefined human threshold value.

The resulting implementation achieved an approximately 30 times speedup over the software baseline while maintaining acceptable human detection capability of 80% under a measured power consumption of 1.715 W.

The rest of the paper is organized as follows. Section II introduces related work surrounding HOG implementations. Section III discusses the methodology and specific optimization strategies used in detail. Section IV describes the validation of the design, and Section V highlights key results and findings. Section VI summarizes the main conclusions.

II. RELATED WORK

Dalal and Triggs [1] introduced HOG as a feature descriptor for human detection by dividing an image window into local cells, accumulating histograms of binned gradient orientations, and normalizing overlapping cell blocks before performing classification with a linear support vector machine (SVM). Their work established the algorithmic foundation for this project and other subsequent implementations. Zhu et al. [2] expanded upon this work by integrating a cascade-of-rejectors

approach with HOG features, using an integral image representation and rejection cascade to significantly speed up gradient magnitude computation. Their work revealed that L1 normalization produces similar detection accuracies compared to traditional square-root-based L2 normalization, which has directly impacted the selection of L1 approximation in this project’s accelerator approach. Zhao et al. [3] proposed an enhanced HOG feature after identifying histogram binning as a limitation in hardware resources and accuracy in discriminative settings. They introduced gradient comparisons against pre-calculated tangent values to eliminate arctangent and a new weighting function that cleans gradients based on their locality.

Kadota et al. [4] was one of the first to implement HOG feature extraction on the FPGA, outlining several methods to reduce computational complexity. They proposed the use of a streaming hardware architecture using line buffers during gradient computation to reduce on-chip memory storage and the simplification of arctangent and square root operations into approximated versions. Bauer et al. [5] implemented a full pedestrian detection system split between an FPGA CPU, and GPU, using optimizations such as fixed-point math, real-time streaming, arctangent approximations, and line buffers to achieve real-time performance with high accuracy.

Later studies evaluated HOG through an architecture-focused approach for further improvement in real-time applications. Mizuno et al. [6] proposed a feature extraction processor featuring a simplified hog algorithm utilizing cell-based scanning and pipelining, simultaneous SVM calculations, and parallelized modules, demonstrating feasibility for real-time SVGA video object detection. Hemmati et al. [7] presented a hardware accelerator for HOG using parallel and deep pipelined architecture with specially defined memory access patterns for improving throughput and lower power while still maintaining high detection accuracy. Their proposed efficient memory access pattern allowed for simultaneous memory access in different functional blocks, avoiding redundant computations at separate pipeline stages. Luo and Lin [8] achieved a pure FPGA implementation combining both HOG feature extraction and SVM classification without external memory or processors. Their proposed design simplified bin assignments and promoted the reuse of cell histogram sums in block normalization, resulting in lower resource utilization but at higher frame rates than Dalal and Triggs.

Other works highlighted resource efficiency and algorithmic simplification with the intention of accelerating the HOG process. Wojek et al. [9] explored parallelizing the sliding window approach on a GPU, allowing for simultaneous processing of independent search windows. Zhou et al. [10] presented a fully pipelined architecture for high throughput in traffic sign classification, utilizing simplified CORDIC-based normalization approximation and parallel SVM dot products for efficient resource matching on an FPGA.

The main distinction of this project is that it combines multiple HLS hardware optimization techniques into a single FPGA-based feature extraction pipeline. Rather than isolating the feature descriptor or specific computational stages, this work evaluates the interactions between modules such as

gradient convolution, histogram generation, and streaming integration to target arithmetic and memory bottlenecks for maximum efficiency within the HOG pipeline, using execution time, speedup, power consumption, and detection accuracy as performance metrics.

III. METHODOLOGY

This project implements and optimizes a hardware-accelerated HOG pipeline for human detection. The baseline algorithm consists of image preprocessing, gradient computation, orientation binning, cell histogram generation, and similarity-based classification. The proposed implementation focuses on accelerating the most computationally expensive feature extraction stages, including gradient convolution and cell histogram generation. These stages were selected based on initial software profiling shown in Figure 1, which revealed that gradient computation accounted for 97.4% of total runtime, followed by the cell histogram stage, which accounted for 2.5%.

TABLE I
SOFTWARE PROFILING RESULTS

Stage	Latency (ms)	Latency (%)
preprocess	0.219 ms	0.0%
convolution	889.853 ms	97.2%
orientation	2.419 ms	0.3%
cell_histograms	22.745 ms	2.5%
TOTAL	915.236 ms	

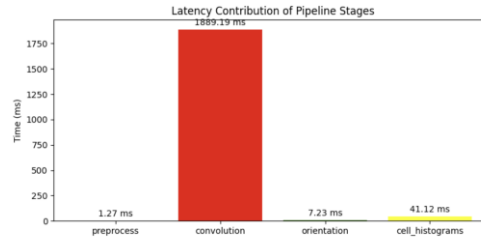


Fig. 1. Initial software profiling results for baseline HOG implementation with the convolution stage as the major bottleneck.

Since software profiling results clearly indicate that gradient convolution is the dominant bottleneck, optimizing it has the greatest potential to reduce overall execution time. This result is expected because gradient convolution requires repeated neighborhood access for each pixel in the image. A direct software implementation repeatedly loads neighboring pixels, which increases memory traffic and causes the convolution stage to dominate runtime.

The profiling results also show that cell histogram generation is the second most important optimization target. Although histogram generation accounts for a much smaller percentage

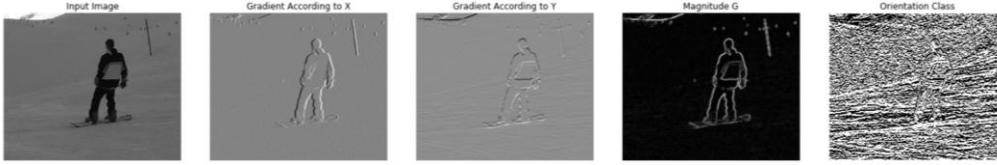


Fig. 2. Intermediate stages of the HOG algorithm. The leftmost image is the image on which HOG will be computed. Subsequent images depict the gradient according to the x- and y-directions, magnitude of the gradient, and the gradient orientation classification.

of total software runtime than gradient convolution, it introduces hardware-specific challenges because multiple pixels may update the same histogram bin. Therefore, the final hardware design targeted both gradient convolution and histogram generation. This is important because optimizing only the largest software bottleneck does not guarantee an efficient full hardware pipeline if later stages introduce memory contention or pipeline stalls.

A. Baseline Pipeline

The baseline software pipeline first converts each input image into a fixed-size grayscale representation and resizes each image to 500x500 or smaller pixels. For each pixel, horizontal and vertical gradients are computed using spatial convolution kernels. These gradient values are then used to estimate the local edge magnitude and orientation. Then, the image is divided into cells of 30x30 pixels, and each cell accumulates a 9-binned histogram of gradient orientations. Cell histograms across the entire image are accumulated to form a feature vector, which is compared against a reference feature vector from an image containing a human. Cosine similarity scores between images are then compared to a threshold value to determine whether an image contains a human.

B. Gradient Convolution Optimization

Gradient convolution was identified as the primary bottleneck in the software implementation because each output pixel depends on a local neighborhood of input pixels. A direct implementation repeatedly accesses neighboring pixels from external memory, which increases memory traffic and limits throughput. To reduce this cost, the hardware design uses a rolling line buffer that stores previous image rows on chip. As the convolution window moves across the image, pixels are reused from the line buffer instead of being repeatedly loaded from external memory.

The optimized convolution module maintains a local window around each pixel and updates this window as new pixels stream through the design. After the line buffer is filled, the module can process a new pixel each clock cycle under a pipelined schedule. This improves throughput by exploiting spatial data reuse and reducing redundant memory accesses. The line buffer and window update logic are also unrolled where appropriate to reduce initialization overhead and maximize pipeline efficiency.

C. Orientation Binning and Magnitude Approximation

After horizontal and vertical gradients are computed, each pixel must be assigned to an orientation bin. Common software implementations often use trigonometric functions such as

inverse tangent to compute the exact gradient angle. However, these operations are expensive in hardware and can increase latency and resource utilization. To make orientation binning more efficient, this project replaces trigonometric angle computation with integer comparison-based binning. Nine predefined orientation bins divide the unsigned angle range from 0° to 180° , and to preserve the unsigned range, both gradients are negated when in the lower half of the plane, reflecting the orientation without changing gradient direction.

Instead of directly computing the angle, the design compares scaled ratios of the vertical and horizontal gradient components against precomputed threshold values. This allows the orientation bin to be determined by using cross-multiplication and comparison rather than division or trigonometric functions. In HLS, the comparison chain synthesizes as a priority encoder and resolves without additional clock cycles. The gradient magnitude is also approximated using the L1-norm, or sum of absolute gradient values, rather than the traditional L2-norm, or square-root-based Euclidean magnitude. This approximation reduces computational complexity while preserving the relative strength of local edges for histogram accumulation with minimal effects on accuracy. The gradient magnitude is a physical representation of how strong light changes at the pixel in the image.

D. Histogram Generation Optimization

The cell histogram stage accumulates gradient magnitudes into orientation bins for each image cell. In hardware, this stage can become a bottleneck because multiple pixels may attempt to update the same histogram memory, creating a BRAM read-modify-write hazard that introduces stalls and reduces pipeline throughput. To combat this limitation, the proposed design uses histogram privatization. Multiple private copies of the histogram are maintained in separate BRAM banks, each partitioned completely for simultaneous access, so that updates can occur without waiting for reads or writes to finish. After all relevant pixels are processed, the private histograms are reduced into a final histogram for each cell in a parallel adder tree that finishes all additions in one cycle.

The design also flattens the inner cell loading loop and histogram reduction loop to improve pipeline scheduling. Flattening reduces nested-loop overhead and helps the hardware maintain a lower initiation interval during histogram computation. Together, histogram privatization and loop restructuring improves parallelism in the cell histogram stage and reduce the performance penalty associated with repeated bin updates.

E. Dataflow Streaming and Tile Buffering

To further improve throughput, the optimized design connects gradient computation and histogram generation using HLS dataflow streaming. In the non-streaming version, gradient convolution and histogram generation execute sequentially, requiring intermediate gradient data to be written to and read from memory. In the streaming version, gradient outputs are passed directly through a first-in, first-out (FIFO) buffer so that histogram generation can begin as soon as gradient values become available. This allows the two stages to overlap in execution.

After adding streaming, a warm-up phase is required inside the gradient convolution loop because the rolling line buffer must first receive enough pixels to form a valid convolution window. The addition of a separate stage divides the buffer priming from the output generation to avoid stalling the downstream histogram stage. In addition, a tile buffer is needed within the histogram generation stage to convert the raster-order gradient stream into a locally accessible region needed for cell histogram computation. The tile buffer loads a cell height number of rows from the stream into a 2D buffer before histogram processing occurs. Once fully loaded, the histogram stage can access pixels in the spatial pattern required for cell-based accumulation without access dependencies, allowing for full use of the throughput benefits provided by streaming.

F. Bit-Width Reduction and Hardware Integration

Bit-width reduction was applied across the convolution-orientation-histogram pipeline to reduce unnecessary hardware resource usage. Since image pixels, gradients, magnitudes, and histogram values have bounded ranges, each data type was assigned a width large enough to represent the maximum expected value without using excessive logic or memory. This reduces resource consumption and can improve timing by avoiding unnecessarily wide arithmetic units.

The accelerator was implemented using C++ inside the Vitis HLS tool for a software-programmable FPGA. The design uses memory-mapped interfaces for image input and output buffers and control registers for runtime parameters such as image dimensions. The final implementation combines rolling line buffering, orientation approximation, histogram privatization, dataflow streaming, tile buffering, and bit-width reduction into a single optimized HOG feature extraction pipeline.

IV. EVALUATION AND VALIDATION

The proposed HOG accelerator was evaluated by comparing a software baseline on an Intel Core i9 CPU against a hardware implementation on a TUL PYNQ-Z2 development board. The PYNQ-Z2 board features an AMD Xilinx Zynq-7000 System-on-Chip (SoC), integrating a dual-core ARM Cortex-A9 processor with programmable FPGA fabric. The software baseline included all HOG stages (preprocessing, gradient convolution, orientation computation, cell histogram creation, similarity comparison and classification) implemented in Python on Jupyter Notebook. The hardware implementation offloaded the optimized convolution and histogram stages to the FPGA and connected the accelerated HOG pipeline with the remaining preprocessing and classification stages through the

Python Productivity for Zynq (PYNQ) framework via AXI4-Lite registers and AXI Direct Memory Access (DMA) buffers.

A. Dataset Formation

The images used for accuracy testing were derived from the following sources. The INRIA human dataset [1] (initially from Dalal and Triggs but sourced on Kaggle) provided human images for accuracy training. The Intel Image Classification dataset [11] sourced from Kaggle but initially published by Intel provided the nonhuman images for initial accuracy training. The Common Objects in Context (COCO) dataset [12] provided the mixed human and nonhuman images for classification testing. Each image was resized and converted into a fixed grayscale input representation before feature extraction. The cosine similarity between HOG feature vectors from a training set of 10 human images and 12 nonhuman images against a reference image is first computed. The threshold that produced the highest training-set classification score was then applied to a separate test set of 10 human images and 10 nonhuman images. This separation between threshold selection and final testing was used to reduce bias in the reported accuracy.

B. Performance Metrics

The main performance metrics were execution time, speedup, detection accuracy, and power consumption. Execution time was selected because the primary goal of the project was to reduce the latency of HOG feature extraction. Speedup was computed as the ratio between the software baseline runtime and the hardware runtime, which directly quantified the performance improvement provided by hardware acceleration. Detection accuracy was measured to verify that the optimized hardware pipeline still produced usable human and nonhuman classification results. Power consumption was included because embedded vision systems are often constrained by energy and thermal limits.

C. Validation

Validation was performed in two ways. First, functional validation was used to verify that the hardware pipeline produced feature outputs that could still be used for similarity-based human detection. This was done by computing the cosine similarity between test-image feature vectors and a reference feature vector and applying the selected similarity threshold to determine which images had similar-enough feature vectors as that of an image containing a human. Second, performance validation was performed by comparing hardware execution time against the software baseline under the same image-processing task. This validation approach confirmed both that the accelerator improved runtime and that the optimized implementation preserved functional detection capability.

For repeatability, the evaluation procedure can be summarized as follows. First, collect human and nonhuman test images and resize them to the selected input resolution. Second, convert each image to grayscale and allocate them as the input image buffer. Third, execute the software HOG pipeline and record the total runtime and per-stage profiling results. Fourth, execute the FPGA-accelerated pipeline on the same input images and record the hardware runtime. Fifth, compute speedup as software runtime divided by hardware runtime.

Finally, compute cosine similarity scores, select the best threshold using the training set, and report final classification accuracy on the held-out test set.

V. EXPERIMENTAL RESULTS AND ANALYSIS

A. Gradient Convolution Acceleration

TABLE II
CONVOLUTION OPTIMIZATION HARDWARE PROFILING

Stage	Latency (ms)	Latency (%)
preprocess	3.581 ms	2.4%
convolution	21.585 ms	14.4%
orientation	12.801 ms	8.6%
cell histograms	111.652 ms	74.6%
TOTAL	149.619 ms	

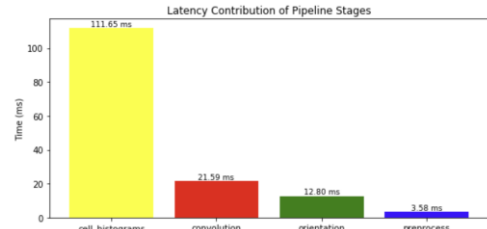


Fig. 3. Profiling results after FPGA-based optimization of the gradient convolution stage demonstrating significant speedup and a new bottleneck of the cell histogram stage.

The optimized gradient convolution stage achieved a runtime reduction from 889.853 milliseconds to 21.585 milliseconds shown in Table II and Figure 3, corresponding to an approximately 41 times speedup, leading to a complete HOG speedup of approximately 6 times. This improvement was primarily caused by the rolling line buffer, which stores previous image rows on the chip and allows each pixel to be reused across multiple convolution windows. In the software baseline, neighboring pixels are repeatedly accessed from memory for each convolution operation. In contrast, the line-buffered hardware design reduces redundant memory reads and supports pipelined processing after the buffer is filled.

This result shows that the gradient stage is well suited for field-programmable gate array acceleration because the computation has a regular spatial access pattern. Once the convolution window is available, the same operation is applied across the image with predictable data reuse. The large speedup also confirms the profiling result: since gradient convolution dominates the software runtime, reducing this stage has a major effect on total pipeline performance.

B. Combined Convolution and Histogram Results

TABLE III
COMBINED OPTIMIZATIONS HARDWARE PROFILING

Stage	Latency (ms)	Latency (%)
preprocess	3.397 ms	28.1%
hog accel	8.700 ms	71.9%
TOTAL	12.097 ms	

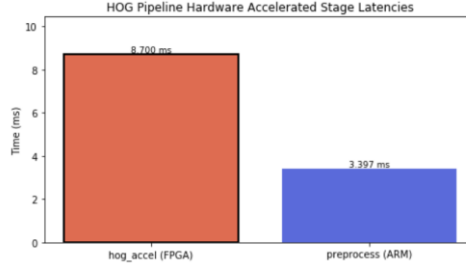


Fig. 4. Final profiling results after implementing all optimizations, including adding on histogram-specific optimizations as well as dataflow streaming to parallelize the convolution and histogram stages.

The completely optimized HOG pipeline combined gradient convolution, orientation computation, and histogram generation through dataflow streaming, eliminating the individual stages from hardware profiling. The combination of optimizations such as the line buffer, histogram privatization, division-free orientation binning, dataflow streaming, bit-width reduction, loop pipelining, and loop unrolling produced a runtime speedup of 105 times from 915.017 milliseconds to 8.7 milliseconds within the accelerated modules. The overall runtime was reduced from 915.236 milliseconds to 12.097 milliseconds, shown in Table III and Figure 4, corresponding to a 75.6 times speedup from the software baseline and additional 12 times speedup since the convolution-only optimizations.

The most significant improvement in efficiency came from dataflow streaming, which allowed the heavily computational convolution and histogram creation stages to be independent hardware processes executing concurrently. The gradient to histogram creation process is streamlined and optimized, allowing efficient parallelization and combined execution time to be the maximum of the two modules instead of the sum of both.

Other important optimizations contributing to the lower latency were histogram privatization and arctangent and square root eliminations. Histogram privatization created separate copies of histograms in different BRAM banks, eliminating the BRAM port-accessing contention between different pixels accessing the same bin. Division-free orientation binning uses precomputed tangent values with cross multiplication to avoid arctangent and an approximated magnitude computation formula to avoid square root, getting rid of multi-cycle calculations and allowing maximum pipeline efficiency.

However, the preprocessing time for the hardware-accelerated module was 15.5 times greater than the original preprocessing time for the software CPU software baseline. This change can be mostly attributed to the software baseline utilizing an Intel Core i9 processor and the hardware implementation using an ARM Cortex-A9 processor on the PYNQ-Z2 board, which allows the software baseline to have much lower processing time than the hardware-accelerated module.

C. Dataset Testing and Accuracy Results



Fig. 4. Cosine similarity accuracy results of a 20-image dataset compared to a reference human image.

The tested dataset resulted in an 80% detection accuracy with two human images wrongly predicted as nonhuman images and two nonhuman images wrongly predicted as human images.

While the detection accuracy is not ideal, it is expected knowing the constraints of the implementation of this project. Our approach computes magnitude with L1 approximation instead of the recommended L2-norm calculation for best accuracy. Our chosen cell size of 30x30 pixels instead of smaller 8x8 pixels also causes a reduction of shape detail when comparing gradients for human edge detection. Standard HOG pipelines utilize block normalization for illumination invariance, which was not incorporated in this project’s version of HOG for simplicity. In addition, cosine similarity is highly dependent on the choice of human reference image and provides no learning on the variation of human shapes compared to the traditional support vector machine (SVM) classifiers described in previous works. However, since the primary intent of this project was to maximize reduction in latency, the 70% detection accuracy demonstrates that the optimized HOG pipeline still provides a functional detection mechanism with a limited training dataset and untraditional classifier.

D. Resource Utilization and Power

TABLE IV
COMBINED OPTIMIZATIONS RESOURCE UTILIZATION

	LUT	FF	DSP	BRAM
Used	3862	5119	22	46
Available	53200	106400	220	140

Based on Vitis HLS implementation reports, the accelerated hardware HOG pipeline uses 7.3% available LUTs, 4.8% available FFs, 10% available DSPs, and 32.9% available BRAM on the PYNQ-Z2 board’s FPGA resources. From Vivado post-implementation power analysis, the hardware-accelerated design has a total on-chip power consumption of 1.715 W. The proposed accelerated HOG pipeline’s low resource usage and power consumption reflect that the design is extremely suitable for real-time processing in resource-constrained environments.

VI. CONCLUSION

This paper proposes an accelerated implementation of the Histograms of Oriented Gradients (HOG) algorithm for human detection. The proposed design targets identified bottlenecks in the feature extraction pipeline, notably gradient convolution,

orientation computation, and cell histogram creation, using rolling line buffers, division-free binning, histogram privatization, bit-width optimization, dataflow streaming and loop pipelining and unrolling. These optimization techniques eliminate redundant memory access, improve parallelization, and reduce resource contention. After offloading the accelerated modules to an FPGA, the experimental results demonstrate an overall speedup of 75.6 times compared to the original software baseline. The optimized pipeline maintains an 80% detection accuracy on a dataset of 20 images with a measured power consumption of 1.715 W. The low-latency, low power consumption, and functionally accurate characteristics of the proposed design suitable make it suitable for power-constrained embedded environments, where real-time speed and energy efficiency are prioritized.

REFERENCES

[1] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2005, pp. 886-893.

[2] Q. A. Zhu, M. C. Yeh, K. T. Cheng, and S. Avidan, "Fast Human Detection Using a Cascade of Histograms of Oriented Gradients," in Proc. of the IEEE International Conference on Computer Vision and Pattern Recognition (CVPR), 2006, pp. II-1491-1498.

[3] Y. Zhao, Y. Zhang, R. Cheng, D. Wei, G. Li, "An Enhanced Histogram of Oriented Gradient for Pedestrian Detection," in IEEE Intelligent Transportation Systems Magazine, vol. 7, no. 3, pp. 29-38, Fall 2015.

[4] R. Kadota, H. Sugano, M. Hiromoto, H. Ochi, R. Miyamoto, and Y. Nakamura, "Hardware Architecture for HOG Feature Extraction," in Proc. Fifth Int. Conf. Intelligent Information Hiding and Multimedia Signal Processing (IIH-MSP), 2009, pp. 1330-1333.

[5] S. Bauer, S. Kohler, K. Doll, and U. Brunsmann, "FPGA-GPU Architecture for Kernel SVM Pedestrian Detection," in Computer Vision and Pattern Recognition Workshops (CVPRW), 2010 IEEE Computer Society Conference on, June 2010, pp. 61-68.

[6] K. Mizuno, Y. Terachi, K. Takagi, S. Izumi, H. Kawaguchi, and M. Yoshimoto, "Architectural Study of HOG Feature Extraction Processor for Real-Time Object Detection," in Proc. IEEE Workshop on Signal Processing Systems (SiPS), 2012, pp. 197-202.

[7] M. Hemmati, M. Biglari-Abhari, S. Berber and S. Niar, "HOG Feature Extractor Hardware Accelerator for Real-Time Pedestrian Detection," 2014 17th Euromicro Conference on Digital System Design, Verona, Italy, 2014, pp. 543-550.

[8] J. H. Luo, and C. H. Lin, "Pure FPGA Implementation of an HOG Based Real-Time Pedestrian Detection System," *Sensors*, vol. 18, no. 4, p. 1174, Apr. 2018.

[9] C. Wojek, G. Dorko, A. Schulz, B. Schiele, "Sliding-Windows for Rapid Object Class Localization: A Parallel Technique," In Deutsche Arbeitsgemeinschaft für Mustererkennung (DAGM), 2008.

[10] Y. Zhou, Z. Chen and X. Huang, "A pipeline architecture for traffic sign classification on an FPGA," 2015 IEEE International Symposium on Circuits and Systems (ISCAS), Lisbon, Portugal, 2015, pp. 950-953.

[11] "Microsoft COCO: Common Objects in Context," in Proc. European Conference on Computer Vision, (ECCV), 2014, pp. 740-755.

[12] P. Bansal, "Intel Image Classification," Kaggle, 2019. [Online]. Available: <https://www.kaggle.com/datasets/punet6060/intel-image-classification>.

Commented [CL1]: Mention resources used??